

## Abstract

This project is a peer-to-peer (P2P) file transfer node built on top of UDP. Each node reads a JSON config that lists its own port and the files held by every peer. Because UDP does not guarantee delivery, the program adds a small reliable transport layer with a three-way handshake, a sliding window using selective repeat, and a single FIN packet to close. Every packet carries a six-byte header that stores a type, a file ID, a sequence number, and a data length, so one socket on one port can serve many transfers at the same time. The node supports at least five transfers in parallel and still completes every file when up to five percent of packets are lost.

## Overview

A node is a single Python process that loads a JSON config file at startup. The config tells the node what port to bind, what files it hosts, and what files each peer hosts. After startup, the node opens a command line where the user can type a file name to download, or type kill to shut down. In the background, one listener thread reads every UDP packet that arrives, which worker threads handle each individual upload or download.

## Packet Format

The packet format in which is implemented in this program is the following:

| type | fileID | seqNum | dataLen | payload |

### Type:

| Type | Name    | Purpose                                                                                                                                                   |
|------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | SYN     | Request which file to transfer, in which the target file is in the payload                                                                                |
| 2    | SYN-ACK | Acknowledgement. This packet's payload is the total chunk count for data requested. This is sent back to the sender to know that the request has arrived. |
| 3    | DATA    | Chunk of the target File, index via seqNum with the chunk bytes in the payload                                                                            |
| 4    | D-ACK   | Acknowledgement. This packet has no payload and is sent back to the sender to know that the "chunk" of data request has arrived.                          |
| 5    | FIN     | Finale, establishes that there are no more information will be sent, as all information has been sent.                                                    |

## FileID:

A unique identifier for each transfer, as it allows multiple transfer information into one port. This helps the computer organize which “information” is which

## seqNum

Useful for DATA transitions, as it allows the program to keep track of each chunk transmitted throughout the network.

## dataLen

Helps finding the payload, as it acts like a guide for indexing. Specifically knows the length of the payload.

## payload

Typically, information being transferred in each Chunk. However, may contain the name of the target file during SYN requests.

## Diagrams

